Project Report - Foundation RBE500 Project

Pranay Katyal, Quinn Ferguson, Lucas Burstein, Mahendran K.

November 2024

1 Phase 2

1.1 Part 1

Velocity Level Kinematics: Implement a node with two services. One takes joint velocities and converts them to end effector velocities, and the second one takes end effector velocities and converts them to joint velocities. For this implementation, first we created a node with two new services in the file "velocity_server.py." Both services use methods stored in the file "kinematic_library.py" for modularity. The first service ("velocity_server") takes in Joint Velocity values, and the Joint Angle values. Using the Homogenous Transformation matrices we have for Forward Kinematics we were able to extract out the Rotation Matrix and Position Vector, which would be later be utilized to build our jacobian matrix.

Since we have 4 Revolute joints we would be getting a Jacobian matrix as :

$$J = \begin{bmatrix} Z_0 \times (O_4 - O_0) & Z_1 \times (O_4 - O_1) & Z_2 \times (O_4 - O_2) & Z_3 \times (O_4 - O_3) \\ Z_0 & Z_1 & Z_2 & Z_3 \end{bmatrix}$$

We can easily get the Z Vector as a product of Rotation matrix R and a column vector

$$\begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

which will give us the Z vector, essentially the 3rd column of the Homogenous transformation matrix, meanwhile we can get the position vector from the 4th column of the homogenous Transformation matrix. Upon building our Jacobian matrix J , we then multiply it with our Joint Velocity Vector $\dot{q}$ that contains the joint velocities :

$$\dot{q} = \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \\ \dot{q}_3 \\ \dot{q}_4 \end{bmatrix}$$

This product gives us our resultant Twist ζ that needs to be applied to get the desired velocities on the joints.

$$\zeta = J * \dot{q}$$

and this Twist can be requested via this Service.

The next Service we created is the Inverse Velocity calculation ("inverse_velocity_server"), where we use the Twist value ζ , and inverse of the jacobian J^{-1} to calculate the respective joint velocities $\dot{q}$.

Since our Jacobian is a Non-Square Matrix, its inverse cannot be found out directly, therefore we proceed by finding its pseudo inverse using one of the inbuilt function in Numpy.

$$np.linalg.pinv()$$

This numpy function helps us by finding the pseudo inverse of the Jacobian matrix, which can be utilized in the equation :

$$J^{-1} * \zeta = \dot{q}$$

to find out the respective joint velocities. Now as long as the joint velocities match the input velocities, that means that we have calculated both of them correctly. The code for defining the services is shown below:

```
import rclpy
from robot_omx import kinematic_library
from rclpy.node import Node
from custom_messages.srv import Velocity, InvVel #This is a custom
    .srv file. Request: Joint angles and velocities as 8 Float32
    variables
                                        # Output: Twist
from sensor_msgs.msg import JointState


class Velocity_Server(Node):
    def __init__(self):
        super().__init__('velocity_server') # Creating a server
            node for calculating end effector velocities
        self.srv = self.create_service(Velocity, 'velocity_server',
             self.velocity_callback) # (Type, name, callback
            function)
        self.inv_srv = self.create_service(InvVel, '
            inverse_velocity_server', self.inv_vel_callback) # (
            Type, name, callback function)
        self.subscription = self.create_subscription(JointState, '
            joint_states', self.listener_callback, 10)
        self.q1 = 0
        self.q2 = 0
        self.q3 = 0
        self.q4 = 0

    def velocity_callback(self, request, response):
        # Extracting joint angles and velocities from request:
        q1 = request.q_1
        q2 = request.q_2
        q3 = request.q_3
        q4 = request.q_4
        q_1_dot = request.q_1_dot
        q_2_dot = request.q_2_dot
        q_3_dot = request.q_3_dot
        q_4_dot = request.q_4_dot

        response.twist = kinematic_library.calc_twist(q1, q2, q3,
            q4, q_1_dot, q_2_dot, q_3_dot, q_4_dot) # Using
            function to calculate twist
```

```
        self.get_logger().info(f'The resulting twist is: {response.
            twist}')

        return response # The results of the velocity kinematics
            are returned

    def inv_vel_callback(self, request, response):
        twist = request.twist

        q_dot_values = kinematic_library.calc_joint_velocities(self
            .q1, self.q2, self.q3, self.q4, twist)

        response.q_1_dot = q_dot_values[0]
        response.q_2_dot = q_dot_values[1]
        response.q_3_dot = q_dot_values[2]
        response.q_4_dot = q_dot_values[3]

        self.get_logger().info(f'The resulting q dot values are: {
            q_dot_values}')

        return response

    def listener_callback(self, message):
        self.q1 = message.position[0]
        self.q2 = message.position[1]
        self.q3 = message.position[2]
        self.q4 = message.position[3]



def main():
    rclpy.init()
    velocity_server = Velocity_Server() # initializing the server
        object
    rclpy.spin(velocity_server)  # Running the node continously
```

The most relevant functions from kinematic_library.py are shown below:

```
#-----------Velocity Kinematics---------------
def get_jacobian(q1, q2, q3, q4):

    DH_table = get_dh_table(q1, q2, q3, q4)
    # First finding relevant transformation matrices:
    H_1_0 = get_transformation_mat(DH_table, 0, 1)
    H_2_0 = get_transformation_mat(DH_table, 0, 2)
    H_3_0 = get_transformation_mat(DH_table, 0, 3)
    H_4_0 = get_transformation_mat(DH_table, 0, 4)

    # Calculating intermediate variables by extracting data from
        the transformation matrices above:
    z_0 = [0, 0, 1]  # by definitions
    z_1 = H_1_0[0:3, 2]
    z_2 = H_2_0[0:3, 2]
    z_3 = H_3_0[0:3, 2]
    o_1 = H_1_0[0:3, 3]
    o_2 = H_2_0[0:3, 3]
    o_3 = H_3_0[0:3, 3]
    o_4 = H_4_0[0:3, 3]

    # Calculating elements of jacobain matrix
    j_v_1 = np.cross(z_0, o_4)
    j_v_2 = np.cross(z_1, o_4 - o_1)
    j_v_3 = np.cross(z_2, o_4 - o_2)
    j_v_4 = np.cross(z_3, o_4 - o_3)
    j_w_1 = z_0
    j_w_2 = z_1
    j_w_3 = z_2
    j_w_4 = z_3
```

```
    # Combingin above elements to form jacobian matrix:
    j_v = np.column_stack((j_v_1, j_v_2, j_v_3, j_v_4))
    j_w = np.column_stack((j_w_1, j_w_2, j_w_3, j_w_4))

    return np.concatenate((j_v, j_w))

#print(get_jacobian(get_dh_table(math.pi/2, math.pi/6, -math.pi/3,
    math.pi/4)))

def calc_twist(q1, q2, q3, q4, q_1_dot, q_2_dot, q_3_dot, q_4_dot):

    jacobian = get_jacobian(q1, q2, q3, q4)

    # Calculating twist from jacobians:
    joint_velocities = [q_1_dot, q_2_dot, q_3_dot, q_4_dot]
    calculated_twist = np.matmul(jacobian, joint_velocities)


    twist = Twist()
    twist.linear.x = calculated_twist[0]
    twist.linear.y = calculated_twist[1]
    twist.linear.z = calculated_twist[2]
    twist.angular.x = calculated_twist[3]
    twist.angular.y = calculated_twist[4]
    twist.angular.z = calculated_twist[5]

    return twist


def calc_joint_velocities(q1, q2, q3, q4, twist):

    #Calculate jacobian from given q values
    j = get_jacobian(q1, q2, q3, q4)

    #Calculate the pseudo inverse jacobian
    pinv = np.linalg.pinv(j)

    #Extract the twist into vector form
    twist_vec = [
        twist.linear.x,
        twist.linear.y,
        twist.linear.z,
        twist.angular.x,
        twist.angular.y,
        twist.angular.z
    ]

    #Calculate q velocities
    q_dots = np.matmul(pinv, twist_vec)

    return q_dots
```

1.2 Part 2

In this section, we describe the implementation of a node designed to incrementally update joint position references for the robot based on the given velocity inputs. This node ensures smooth control of motion by integrating velocity commands into position updates. The equation governing the updates is given by:

$$q_{\text{ref}} = q_{\text{ref, old}} + \Delta q \cdot t_{\text{sampling}}$$

where:

- q_{ref}: Updated joint position reference vector.
- $q_{\text{ref, old}}$: Previous joint position reference vector.
- Δq: Joint velocity vector (obtained from the velocity control node).
- t_{sampling}: Fixed sampling time for updates.

This node interacts with the robot's joint position controllers, acting effectively as a velocity controller.

Node Implementation : The node was implemented in Python using the ROS2 framework in the file "robot_node.py." The following code snippet illustrates the implementation:

```
import rclpy
from robot_omx import kinematic_library
from rclpy.node import Node
from geometry_msgs.msg import Pose, Twist
from custom_messages.srv import InvKin, InvVel
from sensor_msgs.msg import JointState # We will need to test this
    in person
from open_manipulator_msgs.srv import SetJointPosition,
    SetKinematicsPose
import time
import csv

class Robot(Node):
    def __init__(self):
        super().__init__('Gripper_Robot')

        self.subscription = self.create_subscription(JointState, '
            joint_states', self.listener_callback, 10)
        self.pose_sub = self.create_subscription(Pose, '
            EndAffectorPose', self.pose_callback, 10)

        self.cli = self.create_client(InvKin, 'inverse_server')
        self.inv_vel_client = self.create_client(InvVel, '
            inverse_velocity_server')
        self.goal_joint_space = self.create_client(SetJointPosition
            , 'goal_joint_space_path')

        self.curr_pose = Pose()
        while not self.cli.wait_for_service(timeout_sec=1.0): #
            Every second it checks to see if the server is
            available
                self.get_logger().info('The␣inverse␣kinematics␣
                    service␣is␣not␣available␣yet.␣Will␣try␣again␣
                    every␣second')
        while not self.inv_vel_client.wait_for_service(timeout_sec
            =1.0): # Every second it checks to see if the server is
             available
                self.get_logger().info('The␣inverse␣velocity␣
                    service␣is␣not␣available␣yet.␣Will␣try␣again␣
                    every␣second')
        while not self.goal_joint_space.wait_for_service(
            timeout_sec=1.0): # Every second it checks to see if
            the server is available
                self.get_logger().info('The␣joint␣setting␣service␣
                    is␣not␣available␣yet.␣Will␣try␣again␣every␣
                    second')

    def move_to_pose(self, pose):
        req = InvKin.Request()
```

```
    req.pose.position.x = pose[0]
    req.pose.position.y = pose[1]
    req.pose.position.z = pose[2]
    req.pose.orientation.x = pose[3]
    req.pose.orientation.y = pose[4]
    req.pose.orientation.z = pose[5]
    req.pose.orientation.w = pose[6]


    self.cli.call_async(req)

def listener_callback(self, msg):
    self.joint_values = msg.position

def set_velocity(self, twist):
    print(f'The␣twist␣received␣is␣{twist}\n\n')
    i = 0
    interval = 0.1
    with open('positions.csv', 'w', newline='') as csvfile:
        file = csv.writer(csvfile)
        file.writerow(["Time", "X", "Y", "Z"])

        rclpy.spin_once(self)
        joint_values = self.joint_values
        time_elapsed = 0

        while(i < 100000):

            req = InvVel.Request()
            req.twist = twist
            response = self.inv_vel_client.call_async(req)
            rclpy.spin_until_future_complete(self, response)  #
                 Ensures program waits for a result prior to
                printing to the terminal.
            joint_velocities = response.result()  # Posting
                result
            time.sleep(interval)

            q_dot_vec = [joint_velocities.q_1_dot,
                joint_velocities.q_2_dot, joint_velocities.
                q_3_dot, joint_velocities.q_4_dot]

            joint_values = self.update_position(q_dot_vec,
                interval, joint_values)
            rclpy.spin_once(self)

            i = i + 1
            time_elapsed+=interval
            print(f'The␣x␣position␣is␣{self.curr_pose.position.
                x}\n')
            file.writerow([time_elapsed, self.curr_pose.
                position.x, self.curr_pose.position.y, self.
                curr_pose.position.z])
    return 0


def update_position(self, velocities, interval,
    initial_joint_values):
    req = SetJointPosition.Request()
    new_joint_values = []


    for i in range(len(velocities)):
        new_joint_values.append(initial_joint_values[i] +
            velocities[i]*interval)
```

```
        print(f'The␣joint␣angles␣are␣now:␣{new_joint_values}\n')
        # Set request variable including joint names, joint values,
             and path time
        req.joint_position.joint_name = ['joint1', 'joint2', '
            joint3', 'joint4', 'gripper']
        new_joint_values.append(0.0)
        req.joint_position.position = new_joint_values
        req.path_time = interval


        # make the request and return failure if it fails
        try:
            self.goal_joint_space.call_async(req)
        except Exception as e:
            self.get_logger().info('Sending␣Goal␣Joint␣failed␣%r' %
                (e,))
        return new_joint_values

    def pose_callback(self, msg):
        self.curr_pose = msg
        print(f'The␣msg␣{msg}\n')




def main():
    rclpy.init()
    rob = Robot()
    time.sleep(3)
    input_twist = Twist()
    input_twist.linear.y = 50.0

    rob.set_velocity(input_twist)
```

Node Working : The workflow of the node is as follows:

1. The node calls the velocity service, which returns the velocity vector Δq.
2. On receiving the velocity data, the node computes the updated joint position references using the formula $q_{\text{ref}} = q_{\text{ref, old}} + \Delta q \cdot t_{\text{sampling}}$.
3. The updated joint positions are sent to the robots goal_joint_space service, which updates the robots position.

Observations : During testing, the incremental position node successfully updated the joint position references at a consistent sampling rate. The following observations were made:

- Using a sampling time of $t_{\text{sampling}} = 0.1$ seconds resulted in approximately smooth motion.
- Any significant deviations were minimized by fine-tuning the sampling time and ensuring reasonable velocity inputs.

1.3 Part 3

The goal of this part is to move the robot on a straight path along the $+y$ direction in Cartesian space. This was achieved by providing a constant Cartesian velocity reference, and feeding it into the node described in part 2.

Implementation: To execute this task, the following steps were performed:

1. A constant Cartesian velocity $\zeta = [0, v_y, 0, 0, 0, 0]^T$ (linear velocity in $+y$) was defined.

2. The Jacobian matrix J, constructed as described in Part 1, was used to calculate the corresponding joint velocities $\dot{q}$:

$$\dot{q} = J^+ \zeta$$

   where J^+ is the pseudo-inverse of the Jacobian matrix.

3. These joint velocities were used to update the position incrementally (Part 2) to move the robot in a straight path.

Code Implementation : The twist was sent to the nodes described in parts 1 and 2 to achieve linear motion, using the code below:

```
def main():
    rclpy.init()
    rob = Robot()
    time.sleep(3)
    input_twist = Twist()
    input_twist.linear.y = 50.0
    rob.set_velocity(input_twist)
```

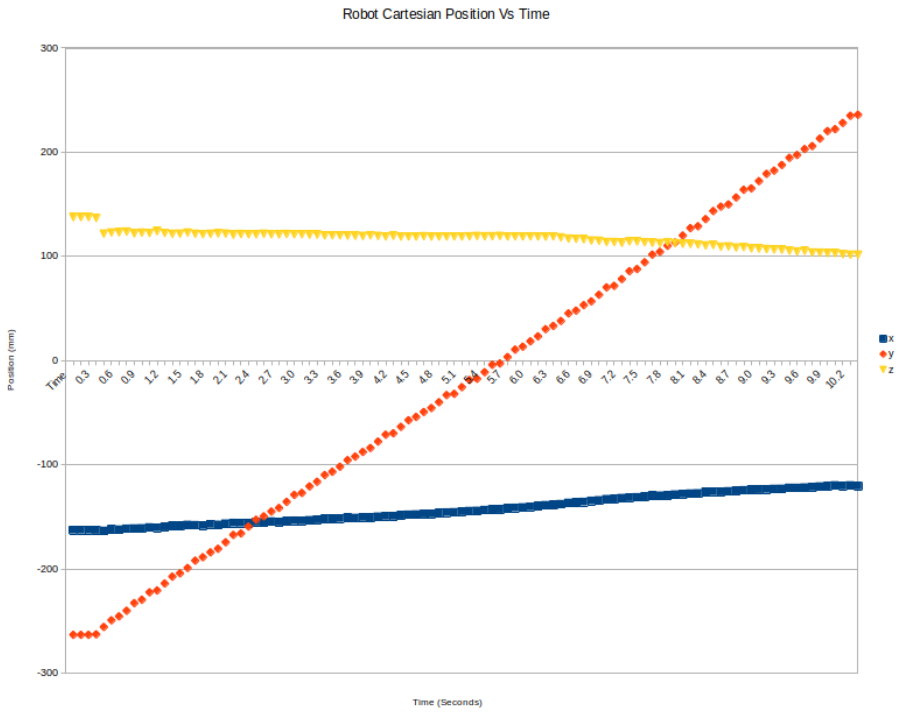
Results : The robot successfully moved along a straight path in the $+y$ direction in Cartesian space.

Observations :

- **Trajectory Accuracy**: The recorded Cartesian positions closely followed the desired straight-line path. Minor deviations were observed due to numerical approximations in the Jacobian pseudo-inverse.

- **Graph**: Results are plotted in the graph below. As can be seen, there was approximately linear motion in the y direction, with minimal movement in the x and z directions.

- **Video**: See the video file submitted with the assignment for an example of the robot moving linearly in the y-direction.


Robot Cartesian Position Vs Time
300
200
100
0
-100
-200
-300
Position (mm)
Time
0.3
0.6
0.9
1.2
1.5
1.8
2.1
2.4
2.7
3.0
3.3
3.6
3.9
4.2
4.5
4.8
5.1
5.4
5.7
6.0
6.3
6.6
6.9
7.2
7.5
7.8
8.1
8.4
8.7
9.0
9.3
9.6
9.9
10.2
Time (Seconds)
x
y
z